Supplementary Material
Adapting Execution-Time Objectives for Multi-Robot Policies via Collaborative Flow Policy Guidance

Williard Joshua Jose, Yuhao Li, Zihao Deng, and Hao Zhang
Human-Centered Robotics Lab, University of Massachusetts Amherst
{wjose, yuhaoli, zihaodeng, hao.zhang}@umass.edu

CONTENTS

I. List of Symbols and Definitions

TABLE I
List of Symbols

Symbol	Description
N	Number of robots in the team
$\mathbf{s}^t$	Global state of the system at time t
$\mathbf{o}_i^t$	Local observation for robot i
$\mathbf{a}_i^t$	Action (control input) for robot i
π_θ	Policy actor network
$\hat{A}^t$	Generalized Advantage Estimate
τ	Flow time variable, $\tau \in [0, 1]$
v_θ	Velocity field
J^{nom}	Nominal team objective (training reward)
J^{exec}	Aggregated execution-time objective
J_m^{exec}	The m-th individual execution cost function
$\boldsymbol{\lambda}$	Preference weights for execution objectives
$\mathbf{a}_\tau$	Noisy action during flow generation
$\hat{\mathbf{a}}(\mathbf{a}_\tau, \tau)$	Estimated terminal action from $\mathbf{a}_\tau$
$\mathbf{g}^{\text{exec}}$	Harmonized gradient of execution objectives
$\mathbf{g}^{\text{nom}}$	Gradient of the nominal flow (velocity vector)
$\rho_t(\theta)$	Probability ratio $\exp(\mathcal{L}_{\text{old}}^{\text{CFM}} - \mathcal{L}^{\text{CFM}})$
$\mathcal{L}^{\text{CFM}}$	Conditional Flow Matching Loss
$\mathcal{L}^{\text{MAFPO}}$	Multi-Agent Flow Policy Optimization Loss
ϵ	Regularization coefficient for SPO

II. Theoretical Analysis and Proofs

A. Proof of Proposition 1 (MAFPO Surrogate Consistency)

Proposition 1. *Minimizing the MAFPO surrogate objective $\mathcal{L}^{MAFPO}(\theta_i)$ yields a policy update direction that maximizes the advantage-weighted Evidence Lower Bound (ELBO). Specifically, $\nabla_{\theta_i}\mathcal{L}^{MAFPO} = -\mathbb{E}[\hat{A}^t \nabla_{\theta_i} ELBO(\mathbf{a}_i|\mathbf{o}_i)]$. Because the ELBO is a variational lower bound on the log-likelihood, this update approximately maximizes the expected joint return $J(\boldsymbol{\theta})$.*

Proof: We analyze the gradient of the MAFPO objective, which integrates the FPO ratio estimator [1] with the SPO regularization [2].

First, we establish the relationship between the Conditional Flow Matching (CFM) loss and the Evidence Lower Bound (ELBO). As derived in [1], [3], the expected CFM loss corresponds to the negative ELBO up to a constant:

$$\mathbb{E}[\mathcal{L}^{\text{CFM}}(\theta)] = -\text{ELBO}(\theta) + C \implies \nabla_\theta \mathcal{L}^{\text{CFM}} = -\nabla_\theta \text{ELBO}. \tag{1}$$

Note that the $\text{ELBO}_\theta(\mathbf{a}|\mathbf{o}) = \log \pi_\theta(\mathbf{a}|\mathbf{o}) - \mathcal{D}_\theta^{\text{KL}}$, where $\mathcal{D}_\theta^{\text{KL}}$ is the KL divergence between the ELBO and likelihood, and the underlying probabilistic model is the stochastic policy $\pi_\theta(\mathbf{a}|\mathbf{o})$ (represented by sampling from the flow model $v_\theta(\mathbf{a}|\mathbf{o})$).

Next, we approximate the probability ratio $\rho_t(\theta_i)$. Substituting the intractable log-likelihoods with their ELBO proxies:

$$\rho_t(\theta_i) = \frac{\pi_{\theta_i}}{\pi_{\theta_i^{\text{old}}}} = \exp\left(\log \pi_{\theta_i} - \log \pi_{\theta_i^{\text{old}}}\right) \tag{2}$$

$$\approx \exp\left(\text{ELBO}(\theta_i) - \text{ELBO}(\theta_i^{\text{old}})\right) \tag{3}$$

$$= \exp\left(\mathcal{L}^{\text{CFM}}(\theta_i^{\text{old}}) - \mathcal{L}^{\text{CFM}}(\theta_i)\right). \tag{4}$$

The MAFPO loss for agent i with SPO regularization is defined as:

$$\mathcal{L}^{\text{MAFPO}}(\theta_i) = -\hat{\mathbb{E}}_t \left[\hat{A}^t \rho_t(\theta_i) - \frac{|\hat{A}^t|}{2\epsilon}(\rho_t(\theta_i) - 1)^2 \right]. \tag{5}$$

We compute the gradient with respect to θ_i. Differentiating term-by-term:

$$\nabla_{\theta_i}\mathcal{L}^{\text{MAFPO}} = -\hat{\mathbb{E}}_t \left[\hat{A}^t \nabla_{\theta_i}\rho_t - \frac{|\hat{A}^t|}{\epsilon}(\rho_t - 1)\nabla_{\theta_i}\rho_t \right]. \tag{6}$$

We evaluate the gradient at the current policy parameters $\theta_i \approx \theta_i^{\text{old}}$, approximating the update direction within the local neighborhood of θ_i^{old}, due to the regularization of the policy. At this point, $\rho_t(\theta_i^{\text{old}}) \approx 1$, causing the regularization gradient to

vanish:

$$\nabla_{\theta_i}\left[\frac{|\hat{A}^t|}{2\epsilon}(\rho_t-1)^2\right]_{\theta_i^{\text{old}}} = \left[\frac{|\hat{A}^t|}{\epsilon}(\rho_t-1)\nabla_{\theta_i}\rho_t\right]_{\theta_i^{\text{old}}} \tag{7}$$

$$= \frac{|\hat{A}^t|}{\epsilon}(1-1)\nabla_{\theta_i}\rho_t\big|_{\theta_i^{\text{old}}} = 0. \tag{8}$$

Then, the gradient of the total loss simplifies to:

$$\nabla_{\theta_i}\mathcal{L}^{\text{MAFPO}}\big|_{\theta_i^{\text{old}}} = -\hat{\mathbb{E}}_t\left[\hat{A}^t\nabla_{\theta_i}\rho_t - 0\right]_{\theta_i^{\text{old}}} \tag{9}$$

$$= -\hat{\mathbb{E}}_t\left[\hat{A}^t\nabla_{\theta_i}\rho_t(\theta_i)\big|_{\theta_i^{\text{old}}}\right]. \tag{10}$$

The gradient of the ratio at θ_i^{old} is:

$$\nabla_{\theta_i}\rho_t\big|_{\theta_i^{\text{old}}} = \nabla_{\theta_i}\exp\left(\mathcal{L}^{\text{CFM}}(\theta_i^{\text{old}}) - \mathcal{L}^{\text{CFM}}(\theta_i)\right)\Big|_{\theta_i^{\text{old}}} \tag{11}$$

$$= 1\cdot(-\nabla_{\theta_i}\mathcal{L}^{\text{CFM}}(\theta_i)). \tag{12}$$

Substituting this back into the MAFPO gradient, we then use the ELBO relationship:

$$\nabla_{\theta_i}\mathcal{L}^{\text{MAFPO}} = -\hat{\mathbb{E}}_t\left[\hat{A}^t(-\nabla_{\theta_i}\mathcal{L}^{\text{CFM}})\right] \tag{13}$$

$$= -\hat{\mathbb{E}}_t\left[\hat{A}^t\nabla_{\theta_i}\text{ELBO}(\mathbf{a}_i|\mathbf{o}_i)\right]. \tag{14}$$

Therefore, minimizing $\mathcal{L}^{\text{MAFPO}}$ performs gradient ascent on the advantage-weighted ELBO.

Finally, we generalize to the multi-agent setting similar to MAPPO [4]. Assuming parameter sharing across N homogeneous agents, the total gradient is the sum of individual gradients:

$$\nabla_\theta\mathcal{L}^{\text{MAFPO}}_{\text{total}} = \sum_{i=1}^{N}\nabla_\theta\mathcal{L}_i^{\text{MAFPO}}(\theta) \approx -\hat{\mathbb{E}}_t\left[\sum_{i=1}^{N}\hat{A}^t\nabla_\theta\text{ELBO}(\mathbf{a}_i|\mathbf{o}_i)\right]. \tag{15}$$

■

B. Proof of Theorem 1 (Guided Flow Regularized Optimization)

Theorem 1. *The guided velocity field, defined by*

$$\tilde{v}_\tau = v_\theta(\mathbf{a}_\tau,\tau) - \eta\nabla_{\mathbf{a}_\tau}J^{exec}(\hat{a}(\mathbf{a}_\tau,\tau)) \tag{16}$$

generates trajectories that approximate the descent direction of a time-dependent regularized objective

$$\mathcal{L}_\tau(\mathbf{a}_\tau) = J^{exec}(\hat{a}(\mathbf{a}_\tau,\tau)) - \frac{1}{\eta}\log p_\tau(\mathbf{a}_\tau) \tag{17}$$

where p_τ represents the flow-induced marginal density and η is the guidance scale.

Proof: We derive the relationship between the guided velocity field $\tilde{v}_\tau$ and the regularized objective $\mathcal{L}_\tau$. Following the framework of general guidance for flow matching [5], the guided vector field is constructed by augmenting the nominal field v_θ with a guidance term g_τ:

$$\tilde{v}_\tau(\mathbf{a}_\tau) = v_\theta(\mathbf{a}_\tau,\tau) + g_\tau(\mathbf{a}_\tau). \tag{18}$$

For optimal transport conditional flow matching with Gaussian paths [6], the nominal velocity field v_θ is proportional to the score function of the marginal density p_τ:

$$v_\theta(\mathbf{a}_\tau,\tau) \approx \nabla_{\mathbf{a}_\tau}\log p_\tau(\mathbf{a}_\tau). \tag{19}$$

Substituting the definition of the guidance term from Eq. (16), where $g_\tau = -\eta\nabla_{\mathbf{a}_\tau}J^{\text{exec}}$, into the expression for $\tilde{v}_\tau$:

$$\tilde{v}_\tau = v_\theta(\mathbf{a}_\tau,\tau) - \eta\nabla_{\mathbf{a}_\tau}J^{\text{exec}}(\hat{a}(\mathbf{a}_\tau,\tau)) \tag{20}$$

$$\approx \nabla_{\mathbf{a}_\tau}\log p_\tau(\mathbf{a}_\tau) - \eta\nabla_{\mathbf{a}_\tau}J^{\text{exec}}(\hat{a}(\mathbf{a}_\tau,\tau)). \tag{21}$$

We factor out $-\eta$:

$$\tilde{v}_\tau \approx -\eta \left(\nabla_{\mathbf{a}_\tau} J^{\text{exec}}(\hat{a}(\mathbf{a}_\tau, \tau)) - \frac{1}{\eta} \nabla_{\mathbf{a}_\tau} \log p_\tau(\mathbf{a}_\tau) \right) \tag{22}$$

$$= -\eta \nabla_{\mathbf{a}_\tau} \left(J^{\text{exec}}(\hat{a}(\mathbf{a}_\tau, \tau)) - \frac{1}{\eta} \log p_\tau(\mathbf{a}_\tau) \right). \tag{23}$$

Comparing this to the definition of $\mathcal{L}_\tau(\mathbf{a}_\tau)$ in Eq. (17):

$$\tilde{v}_\tau \approx -\eta \nabla_{\mathbf{a}_\tau} \mathcal{L}_\tau(\mathbf{a}_\tau). \tag{24}$$

Thus, $\tilde{v}_\tau$ corresponds to a gradient descent step on the regularized objective $\mathcal{L}_\tau$, optimizing the execution cost J^{exec} regularized by the log-likelihood of the learned action distribution (from pretraining the flow model). ■

C. Proof of Proposition 2 (Pareto Descent of Execution Objectives)

Proposition 2. *The harmonized execution gradient $\mathbf{g}^{exec}$ derived via iterative projection lies within the intersection of the descent cones of the individual objectives $\{J_m^{exec}\}_{m=1}^M$, provided such an intersection exists. Consequently, an update along $\mathbf{g}^{exec}$ does not increase any individual cost J_m^{exec} locally.*

Proof: Let $\mathbf{g}_m = -\nabla J_m^{\text{exec}}$ denote the negative gradient (descent direction) for the m-th execution objective. When a conflict is detected between objectives i and j (i.e., $\langle \mathbf{g}_i, \mathbf{g}_j \rangle < 0$), the algorithm projects $\mathbf{g}_i$ onto the normal plane of $\mathbf{g}_j$, following the gradient projection approach proposed in [7] and extended in [8]. The modified gradient $\tilde{\mathbf{g}}_i$ is given by:

$$\tilde{\mathbf{g}}_i = \mathbf{g}_i - \text{Proj}_{\mathbf{g}_j}(\mathbf{g}_i) = \mathbf{g}_i - \frac{\langle \mathbf{g}_i, \mathbf{g}_j \rangle}{\|\mathbf{g}_j\|^2} \mathbf{g}_j \tag{25}$$

We verify that $\tilde{\mathbf{g}}_i$ is still a valid descent direction for its own objective J_i by computing the inner product $\langle \mathbf{g}_i, \tilde{\mathbf{g}}_i \rangle$:

$$\langle \mathbf{g}_i, \tilde{\mathbf{g}}_i \rangle = \left\langle \mathbf{g}_i, \mathbf{g}_i - \frac{\langle \mathbf{g}_i, \mathbf{g}_j \rangle}{\|\mathbf{g}_j\|^2} \mathbf{g}_j \right\rangle \tag{26}$$

$$= \|\mathbf{g}_i\|^2 - \frac{\langle \mathbf{g}_i, \mathbf{g}_j \rangle}{\|\mathbf{g}_j\|^2} \langle \mathbf{g}_i, \mathbf{g}_j \rangle \tag{27}$$

$$= \frac{\|\mathbf{g}_i\|^2 \|\mathbf{g}_j\|^2 - (\langle \mathbf{g}_i, \mathbf{g}_j \rangle)^2}{\|\mathbf{g}_j\|^2} \tag{28}$$

By the Cauchy-Schwarz inequality, $(\langle \mathbf{g}_i, \mathbf{g}_j \rangle)^2 \leq \|\mathbf{g}_i\|^2 \|\mathbf{g}_j\|^2$. Thus:

$$\langle \mathbf{g}_i, \tilde{\mathbf{g}}_i \rangle \geq 0 \tag{29}$$

This shows that the projected gradient is not an ascent direction for the cost J_i.

Next, we verify that $\tilde{\mathbf{g}}_i$ does not conflict with objective j by checking if they are orthogonal:

$$\langle \mathbf{g}_j, \tilde{\mathbf{g}}_i \rangle = \left\langle \mathbf{g}_j, \mathbf{g}_i - \frac{\langle \mathbf{g}_i, \mathbf{g}_j \rangle}{\|\mathbf{g}_j\|^2} \mathbf{g}_j \right\rangle \tag{30}$$

$$= \langle \mathbf{g}_j, \mathbf{g}_i \rangle - \frac{\langle \mathbf{g}_i, \mathbf{g}_j \rangle}{\|\mathbf{g}_j\|^2} \|\mathbf{g}_j\|^2 \tag{31}$$

$$= \langle \mathbf{g}_j, \mathbf{g}_i \rangle - \langle \mathbf{g}_i, \mathbf{g}_j \rangle = 0 \tag{32}$$

Since $\langle \mathbf{g}_j, \tilde{\mathbf{g}}_i \rangle = 0$, the update for objective i will not increase the cost of objective j locally.

The combined update direction is $\mathbf{g}^{\text{exec}} = \sum_m \tilde{\mathbf{g}}_m$. For any specific objective k, the inner product is:

$$\langle \mathbf{g}^{\text{exec}}, \mathbf{g}_k \rangle = \sum_m \langle \tilde{\mathbf{g}}_m, \mathbf{g}_k \rangle \tag{33}$$

Since the projection ensures $\langle \tilde{\mathbf{g}}_m, \mathbf{g}_k \rangle \geq 0$ for all pairs, we find that the total sum is non-negative. Thus, the resulting gradient update will not increase any execution cost. ■

D. Proof of Proposition 3 (Non-Destructive Projection of Execution Guidance to the Collaborative Manifold)

Proposition 3. *Let $\mathbf{g}^{nom} = v_\theta(\mathbf{a}_\tau, \tau)$ be the nominal velocity field and $\mathbf{g}^{exec}$ be the harmonized execution descent direction. The effective update direction $\mathbf{d} = \frac{d\mathbf{a}_\tau}{d\tau} = \mathbf{g}^{nom} + \mathbf{g}^{exec\perp}$, where $\mathbf{g}^{exec\perp}$ is the projection of $\mathbf{g}^{exec}$ with respect to $\mathbf{g}^{nom}$, satisfies $\langle \mathbf{d}, \mathbf{g}^{nom} \rangle \geq \|\mathbf{g}^{nom}\|^2$.*

Proof: We analyze the alignment between the effective update direction $\mathbf{d}$ and the nominal velocity field $\mathbf{g}^{\text{nom}}$ by computing their inner product $\langle \mathbf{d}, \mathbf{g}^{\text{nom}} \rangle$. We note that $\mathbf{g}^{\text{exec}}$ represents the descent direction of the execution objectives, consistent with the subtraction of the gradient in the unprojected ODE update. We consider two cases based on the alignment of $\mathbf{g}^{\text{exec}}$ with $\mathbf{g}^{\text{nom}}$.

Case 1: Non-Conflicting Guidance. Suppose $\langle \mathbf{g}^{\text{exec}}, \mathbf{g}^{\text{nom}} \rangle \geq 0$. In this scenario, the execution objective either aligns with or is orthogonal to the nominal policy. The projection mechanism sets $\mathbf{g}^{\text{exec}\perp} = \mathbf{g}^{\text{exec}}$. We substitute this into the inner product:

$$\langle \mathbf{d}, \mathbf{g}^{\text{nom}} \rangle = \langle \mathbf{g}^{\text{nom}} + \mathbf{g}^{\text{exec}}, \mathbf{g}^{\text{nom}} \rangle \tag{34}$$

$$= \|\mathbf{g}^{\text{nom}}\|^2 + \langle \mathbf{g}^{\text{exec}}, \mathbf{g}^{\text{nom}} \rangle \tag{35}$$

$$\geq \|\mathbf{g}^{\text{nom}}\|^2 \tag{36}$$

Here, the guidance can potentially reinforce the nominal flow, and only strictly preserve the movement along the learned manifold, not against.

Case 2: Conflicting Guidance. Suppose $\langle \mathbf{g}^{\text{exec}}, \mathbf{g}^{\text{nom}} \rangle < 0$. This means the execution objective at least partially opposes the nominal policy. We first check if $\mathbf{g}^{\text{exec}\perp}$ is orthogonal to $\mathbf{g}^{\text{nom}}$:

$$\langle \mathbf{g}^{\text{exec}\perp}, \mathbf{g}^{\text{nom}} \rangle = \langle \mathbf{g}^{\text{exec}}, \mathbf{g}^{\text{nom}} \rangle - \frac{\langle \mathbf{g}^{\text{exec}}, \mathbf{g}^{\text{nom}} \rangle}{\|\mathbf{g}^{\text{nom}}\|^2} \langle \mathbf{g}^{\text{nom}}, \mathbf{g}^{\text{nom}} \rangle = 0 \tag{37}$$

Then we compute the alignment of the effective update:

$$\langle \mathbf{d}, \mathbf{g}^{\text{nom}} \rangle = \langle \mathbf{g}^{\text{nom}} + \mathbf{g}^{\text{exec}\perp}, \mathbf{g}^{\text{nom}} \rangle \tag{38}$$

$$= \|\mathbf{g}^{\text{nom}}\|^2 + \langle \mathbf{g}^{\text{exec}\perp}, \mathbf{g}^{\text{nom}} \rangle \tag{39}$$

$$= \|\mathbf{g}^{\text{nom}}\|^2 \tag{40}$$

■

III. Details of Environment Nominal and Execution Objectives

We provide a detailed description of the environments, the nominal objectives used for training, and the execution objectives used for evaluation. We utilize the Vectorized Multi-Agent Simulator (VMAS) [9], a framework for multi-robot simulation in two dimensions that is able to run directly on GPU for parallelization. All environments operate in a 2D continuous space with physics-based dynamics. The robots are modeled as holonomic agents with state consisting of 2D pose and velocity. The robot actions are force-controlled (hence, proportional to acceleration).

A. Navigation Environment

Environment Description: In the Navigation task, N robots are spawned at random start positions and need to navigate to N randomly assigned goal positions $\mathbf{g}_i$. The environment includes inter-robot collisions. The task is considered successful when all robots are within a threshold distance of their respective goals.

Nominal Objective ($J^{\text{nom}}_{\text{navigation}}$): The nominal policy is trained to minimize the robot travel time to respective goals while avoiding collisions. The dense reward function at each timestep t is defined as:

$$R^t = \sum_{i=1}^{N} \left(-\|\mathbf{p}_i^t - \mathbf{g}_i\|_2 - \lambda_{\text{coll}} \cdot \mathbb{I}(\text{collision}_i) \right) \tag{41}$$

where λ_{coll} is a penalty for collisions. The nominal objective J^{nom} corresponds to minimizing:

$$J^{\text{nom}}(\mathbf{s}, \mathbf{a}) = \mathbb{E}\left[\sum_{t=0}^{T} \sum_{i=1}^{N} \left(\|\mathbf{p}_i^t - \mathbf{g}_i\|_2 + \lambda_{\text{coll}} \cdot \mathbb{I}(\text{collision}_i) \right) \right] \tag{42}$$

Execution Objectives:

1. **Speed Limit Cost** ($J^{\text{exec}}_{\text{speed}}$): Penalizes robots for exceeding a speed limit. We set the limit to $v_{\text{limit}} = 0.4$.

$$J^{\text{exec}}_{\text{speed}}(\mathbf{a}) = w_{\text{speed}} \sum_{i=1}^{N} \text{ReLU}(\|\hat{\mathbf{v}}_i\|_2 - v_{\text{limit}}) \tag{43}$$

2. **Safety Margin Cost** ($J^{\text{exec}}_{\text{margin}}$): Ensures a minimum separation distance $d_{\text{safe}} = 0.4$ between all robot pairs. We weight this cost by the distance to the goal in order to relax safety constraints as robots move closer to their goals. Let $w_i = \text{clamp}(\|\mathbf{p}_i - \mathbf{g}_i\|^2/4, 0.1, 1.0)$.

$$J^{\text{exec}}_{\text{margin}}(\mathbf{a}) = w_{\text{safety}} \sum_{i=1}^{N} \sum_{j \neq i} \min(w_i, w_j) \cdot \text{ReLU}(d_{\text{safe}} - \|\hat{\mathbf{p}}_i - \hat{\mathbf{p}}_j\|_2) \tag{44}$$

where $\hat{\mathbf{p}}_i = \mathbf{p}_i + (\mathbf{v}_i + \mathbf{a}_i \Delta t)\Delta t$ is the estimated next position in 2D space.

B. Balance Environment Objectives

Environment Description: In the Balance task, N robots must cooperatively transport a payload to a target location. The payload is subject to gravity and friction, and the robots must apply forces to move it while keeping it balanced on top of them while pushing it collaboratively. The robots are not attached to the payload and only interact with it through contact forces.

Nominal Objective ($J^{\text{nom}}_{\text{balance}}$): Our training reward encourages the payload to move towards the target position $\mathbf{g}_{\text{payload}}$.

$$R^t = -\|\mathbf{p}^t_{\text{payload}} - \mathbf{g}_{\text{payload}}\|_2 - \lambda_{\text{drop}} \cdot \mathbb{I}(\text{payload dropped}) \tag{45}$$

The nominal objective is to minimize the distance of the payload to the target:

$$J^{\text{nom}}(\mathbf{s}, \mathbf{a}) = \mathbb{E}\left[\sum_{t=0}^{T} \|\mathbf{p}^t_{\text{payload}} - \mathbf{g}_{\text{payload}}\|_2\right] \tag{46}$$

Execution Objectives:

1. **Spatial Order Cost** ($J^{\text{exec}}_{\text{order}}$): Enforces a specific left-to-right ordering of robots based on their IDs (e.g., Robot 1 must be left of Robot 0). Let x_i be the x-coordinate of robot i. For a target permutation $\sigma = [1, 0, 3, 2, 4]$:

$$J^{\text{exec}}_{\text{order}}(\mathbf{a}) = w_{\text{order}} \sum_{k=1}^{N-1} \text{ReLU}((\hat{x}_{\sigma(k)} - \hat{x}_{\sigma(k+1)}) + 0.2) \tag{47}$$

This cost is positive if robot $\sigma(k+1)$ is not sufficiently to the right of robot $\sigma(k)$ (by at least 0.2m).

2. **Spacing Cost** ($J^{\text{exec}}_{\text{spacing}}$): Maintains a specific target distance $d_{\text{target}} = 0.15$m between adjacent robots.

$$J^{\text{exec}}_{\text{spacing}}(\mathbf{a}) = w_{\text{spacing}} \sum_{i=1}^{N} (\min_{j \neq i} \|\hat{\mathbf{p}}_i - \hat{\mathbf{p}}_j\|_2 - d_{\text{target}})^2 \tag{48}$$

C. Sampling Environment Objectives

Environment Description: In the Sampling task, robots explore an environment to gather samples from a 2D field, where the sample density is distributed spatially as a mixture of Gaussians. The sample density $I(\mathbf{p})$ varies across the map. Robots accumulate reward proportional to the density at their current position.

Nominal Objective ($J^{\text{nom}}_{\text{sampling}}$): The team is trained to maximize the total samples collected.

$$R^t = \sum_{i=1}^{N} I(\mathbf{p}^t_i) \tag{49}$$

The nominal objective J^{nom} is defined as the negative total collected samples:

$$J^{\text{nom}}(\mathbf{s}, \mathbf{a}) = \mathbb{E}\left[-\sum_{t=0}^{T} \sum_{i=1}^{N} I(\mathbf{p}^t_i)\right] \tag{50}$$

Execution Objectives:

1. **Geofencing Cost** ($J^{\text{exec}}_{\text{geo}}$): Constrains robots to stay away from a circular region defined by center $\mathbf{c} = [0.2, -0.1]$ and radius $r_{\text{geo}} = 0.3$.

$$J^{\text{exec}}_{\text{geo}}(\mathbf{a}) = w_{\text{geo}} \sum_{i=1}^{N} \text{ReLU}(r_{\text{geo}} - \|\hat{\mathbf{p}}_i - \mathbf{c}\|_2) \tag{51}$$

2. **Clustering Cost** ($J^{\text{exec}}_{\text{cluster}}$): Encourages agents to stay close to the team centroid $\bar{\mathbf{p}} = \frac{1}{N}\sum \hat{\mathbf{p}}_i$, maintaining a radius $r_{\text{cluster}} = 0.2$.

$$J^{\text{exec}}_{\text{cluster}}(\mathbf{a}) = w_{\text{cluster}} \sum_{i=1}^{N} (\|\hat{\mathbf{p}}_i - \bar{\mathbf{p}}\|_2 - r_{\text{cluster}})^2 \tag{52}$$

IV. ALGORITHMS

A. Training Algorithm: Multi-Agent Flow Policy Optimization (MAFPO)

Algorithm 1 details the on-policy training procedure. The algorithm proceeds in iterations; during each iteration, we collect a batch of trajectories by executing the current policy π_θ in the environment. We then use generalized advantage estimation (GAE) to compute advantages based on the rewards and value estimates from the critic. The policy update trains the flow matching model to maximize expected returns.

Algorithm 1: MAFPO Training

Input: Initial parameters θ, ϕ, Env $\mathcal{E}$, Iterations K

for $k = 1$ *to* K **do**
  Collect batch $\mathcal{B} = \{(\mathbf{s}, \mathbf{o}, \mathbf{a}, r, \mathbf{s}')\}$ of size N_{frames} using π_θ in $\mathcal{E}$ `// Data Collection`
  Compute advantages $\hat{A}$ using GAE and v_ϕ;
  `// Policy Update`
  for $e = 1$ *to* E *epochs* **do**
    Shuffle $\mathcal{B}$ and create mini-batches;
    for *each mini-batch* b **do**
      Sample time $\tau \sim \mathcal{U}[0, 1]$ and noise $\mathbf{z} \sim \mathcal{N}(0, I)$;
      Generate flow state $\mathbf{a}_\tau = (1 - \tau)\mathbf{z} + \tau\mathbf{a}$;
      Compute vector field prediction $v_\theta(\mathbf{a}_\tau, \mathbf{o}, \tau)$;
      Compute target vector $u_t = \mathbf{a} - \mathbf{z}$;
      Compute CFM Loss: $\mathcal{L}_{\text{CFM}} = \|v_\theta - u_t\|^2$;
      Compute Ratio: $\rho(\theta) = \exp(\mathcal{L}_{\text{CFM}}^{\text{old}} - \mathcal{L}_{\text{CFM}})$;
      Update θ via MAFPO objective: $\mathcal{L}_{\text{MAFPO}} = -\hat{A}\rho + \frac{|\hat{A}|}{2\epsilon}(\rho - 1)^2$;
      Update Critic v_ϕ by minimizing MSE loss;
    end
  end
end

B. Execution Algorithm: Collaborative Flow Policy Guidance (CFPG)

Algorithm 2 describes the inference process. Starting from a Gaussian noise sample, we iteratively solve the flow ODE over T steps. At each step, the nominal velocity field v_θ is computed using the pretrained model, and the terminal action $\hat{\mathbf{a}}$ is estimated to evaluate the gradients of the execution objectives per step. These gradients are harmonized via hierarchical projection to resolve conflicts among objectives and ensure consistency with the learned collaborative manifold. Finally, the state is updated using the guided velocity field, encouraging the generation of actions that satisfy both the nominal and execution constraints.

V. IMPLEMENTATION DETAILS

A. Software and Hardware Architecture

We discuss the three environments used for the training and execution of CFPG. We provide a summary of our software and hardware architecture in Fig. 1.

1) VMAS Training and Execution Simulation: Our training infrastructure is built on TorchRL and VMAS (Vectorized Multi-Agent Simulator). This stack aligns with BenchMARL, enabling standardized benchmarking for future research.

- Physics Simulation: VMAS utilizes a vectorized and differentiable 2D physics engine written in PyTorch.
- Agent Dynamics: Agents are modeled as omnidirectional robots, consistent with the default implementation in VMAS. The action space controls velocity in x, y, and θ.
- High-Throughput Data Collection: The training configuration uses a batch size of 60,000 frames collected from multiple parallelized environments.

2) High-Fidelity Simulation: For validation in unstructured environments, we utilize a simulator built in Unity.

- Environment: The simulator features outdoor terrain and utilizes the Unity game engine and its integrated physics engine.
- Communication Bridge: A bridge using ROS 1 (Robot Operating System) connects the simulation to the execution node. The simulation publishes robot states as ROS topics, and the execution node publishes velocity commands. This setup tests the transfer of policies trained in the vector simulator to a 3D environment with different physics.

Algorithm 2: CFPG Execution-Time Guidance (Inference)

Input: Observation $\mathbf{o}$, Trained Policy v_θ, Guidance Scale η, Solver Steps T, Cost Functions $\{J_m\}_{m=1}^{M}$
Output: Action $\mathbf{a}_{\text{final}}$

Sample noise $\mathbf{a}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$;
Define time steps $\tau_0, \ldots, \tau_T$ from 0 to 1;
for $k = 0$ *to* $T - 1$ **do**
  Current state $\mathbf{a}_k$, time τ_k;
  Compute nominal flow: $\mathbf{v}_{\text{nom}} = v_\theta(\mathbf{a}_k, \mathbf{o}, \tau_k)$;
  Estimate terminal action: $\hat{\mathbf{a}} = \mathbf{a}_k + (1 - \tau_k)\mathbf{v}_{\text{nom}}$;
  Initialize gradients $\mathcal{G} = [\,]$ `// Compute Gradients`
  for $m = 1$ *to* M **do**
    Compute cost $c = J_m(\hat{\mathbf{a}}, \mathbf{o})$;
    Compute grad $g_m = \nabla_{\mathbf{a}_k} c$;
    $\mathcal{G}$.append(g_m);
  end
  Harmonize execution gradients via Gradient Projection: $g_{\text{exec}} = -\text{Project}(\mathcal{G})$;
  Check alignment: $\rho = g_{\text{exec}} \cdot \mathbf{v}_{\text{nom}}$ `// Manifold Consistency Projection`
  if $\rho < 0$ **then**
    Project g_{exec} onto null space of $\mathbf{v}_{\text{nom}}$;
    $g_{\text{exec}} \leftarrow g_{\text{exec}} - \frac{\rho}{\|\mathbf{v}_{\text{nom}}\|^2}\mathbf{v}_{\text{nom}}$;
  end
  Effective velocity: $\mathbf{v}_{\text{eff}} = \mathbf{v}_{\text{nom}} + \eta \cdot g_{\text{exec}}$;
  Step state (Euler): $\mathbf{a}_{k+1} = \mathbf{a}_k + \mathbf{v}_{\text{eff}} \cdot (\tau_{k+1} - \tau_k)$ `// Update Step`
end
return clamp$(\mathbf{a}_T, -1, 1)$;

3) Physical Robot Deployment: Physical experiments use a team of AgileX LIMO robots.

- Hardware Specs: Execution runs on a laptop equipped with an NVIDIA RTX 4060 GPU, 32GB RAM, and an AMD Ryzen AI 9 HX 370 processor.
- ROS 2 Integration: The control system runs on ROS 2 Humble. A central policy node subscribes to robot poses and publishes velocity commands (/cmd_vel) via Wi-Fi.
- Mixed-Reality Sensing: An OptiTrack Motion Capture system provides pose estimation. An overhead projector visualizes virtual objectives in the physical world, allowing for dynamic objective modification while control logic relies on motion capture data.

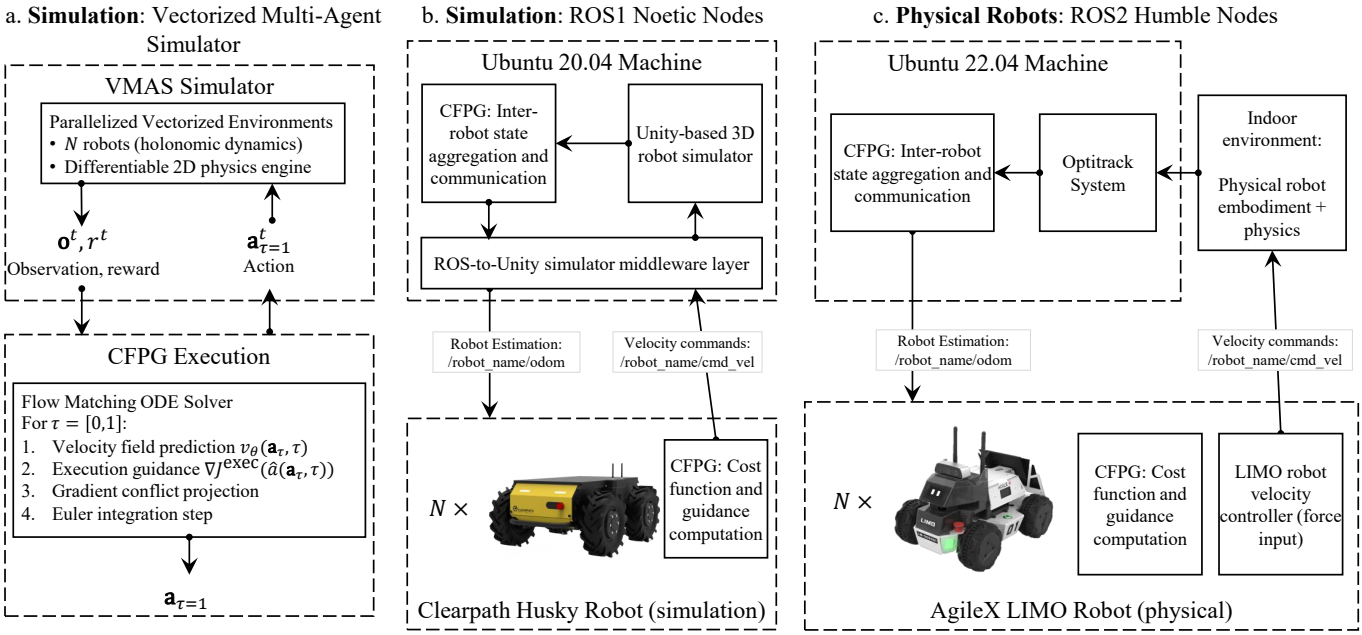

Fig. 1. Software and Hardware Architecture for (a) Physical Robot Deployment, (b) Unity High-Fidelity Simulation, and (c) VMAS Parallelized Training.

B. Network Architecture and Hyperparameters

We implement both the Actor and Critic using Multi-Layer Perceptrons (MLPs) in PyTorch.

- **Actor (v_θ):** The inputs are: observation, action, and a sinusoidal time embedding (dimension 16). MLP hidden layers: [256, 256]. Activation function: SiLU. Output: Action dimension.
- **Critic (v_ϕ):** Input: Global state. MLP hidden layers: [256, 256]. Activation function: SiLU. Output: scalar representing the value of the state.

Training Hyperparameters:

- Learning Rate: 5×10^{-5} (Actor & Critic)
- Batch Size: 4096 (Mini-batch: 256)
- SPO Regularization (ϵ): 0.2
- Entropy Coefficient: 0.0
- Max Gradient Norm: 40.0
- Discount Factor (γ): 0.9
- GAE Parameter (λ): 0.9
- Frames per Batch: 60,000
- Total Training Steps: 30,000,000

C. Training Setup and Hyperparameter Search

To identify the optimal configuration for our Multi-Agent Flow Policy Optimization (MAFPO), we conducted a comprehensive hyperparameter search using a high-performance computing cluster managed by the Slurm workload manager. We utilized Slurm job arrays to efficiently parallelize the training of multiple distinct experimental configurations. The sweep covered the following hyperparameters across all three environments (Navigation, Sampling, Balance):

- **Number of Agents (N):** $\{3, 5, 7\}$ to evaluate scalability across varying robot team sizes.
- **Learning Rate:** $\{1 \times 10^{-4}, 3 \times 10^{-4}, 5 \times 10^{-4}\}$ to determine the optimal step size for convergence.
- **SPO Regularization (ϵ):** $\{0.1, 0.2, 0.3\}$ to balance the stability of the policy update.
- **Solver Steps (T):** $\{3, 6, 10\}$ to analyze the trade-off between flow generation quality and computational cost.

The final hyperparameters used in our main experiments were selected based on the configuration that resulted in the highest mean evaluation reward and model stability during training.

VI. Additional Experiments and Analysis

A. Computational Feasibility Analysis

A primary concern in applying generative models to robotics is inference latency. Unlike Diffusion Policy [10], which typically require $T = 50$ to $T = 100$ denoising steps, flow matching with optimal transport paths learns straight-line trajectories for transforming the action distribution, allowing for significantly fewer steps. In Fig. 2, we investigate the real-world execution frequency of running CFPG execution across different numbers of ODE solution steps. We evaluate the inference speed on both machine learning-specific servers (Intel Xeon CPU with NVIDIA L4 GPU) and embedded hardware used for physical robots (NVIDIA Jetson Orin Nano). We observe that reducing the number of solver steps significantly increases the control frequency but also negatively affects the mean rewards of the flow matching model. This highlights the trade-off between action prediction quality and computation time. We find that at 10 steps, mean rewards start to plateau, while maintaining latencies that allow for control frequencies faster than 10 Hz.

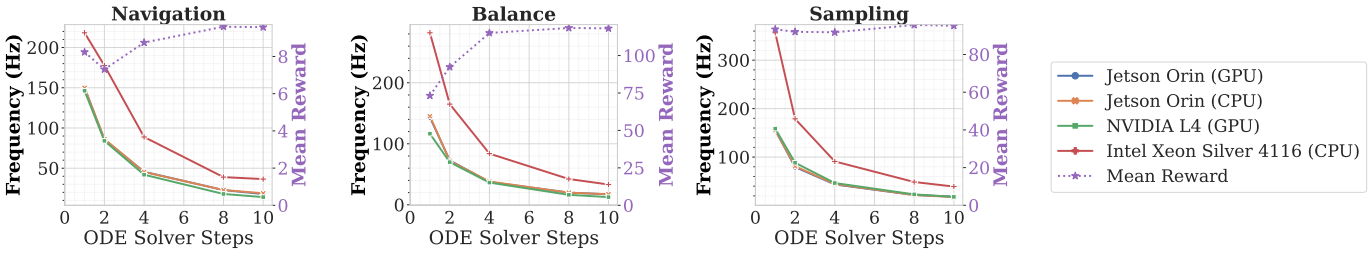

Fig. 2. Computational feasibility analysis showing the trade-off between control frequency (Hz) and task performance (Mean Reward) as a function of ODE solver steps (T). We compare inference speeds on machine learning-specific hardware (Intel Xeon CPU with NVIDIA L4 GPU) and embedded hardware (NVIDIA Jetson Orin Nano).

B. Hyperparameter Analysis on Guidance Strength

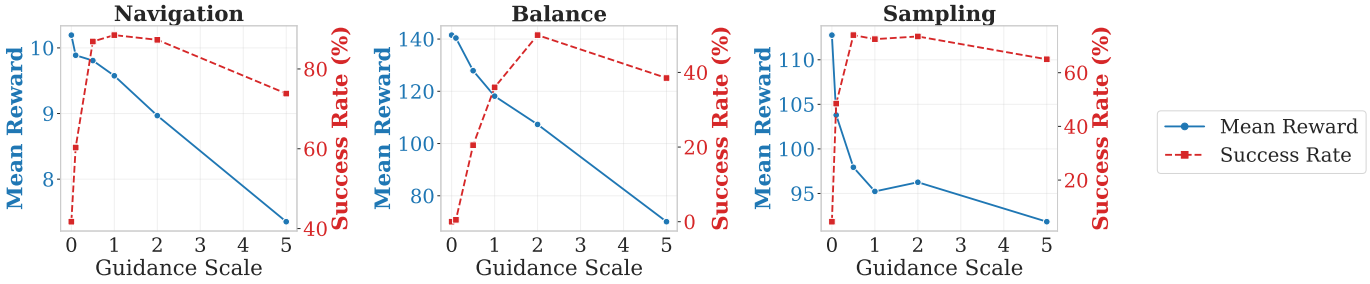

Fig. 3. Hyperparameter sensitivity analysis of guidance scale. We plot the Mean Reward (nominal task performance) and Success Rate (execution objective satisfaction) for Navigation, Balance, and Sampling tasks. Results show a trade-off where moderate guidance improves execution success without degrading nominal performance.

We analyze the performance of CFPG with respect to the magnitude of the guidance strength in Fig. 3. Intuitively, we observe that the lack of guidance prevents our policy from satisfying new execution constraints, resulting in a comparatively low success rate. As we increase the guidance strength, the success rate for execution objectives improves significantly. However, guidance that is too high causes the robots to deviate too far from the learned collaborative policy, which leads to lower mean rewards for the nominal objective. This verifies the importance of the hierarchical gradient projection and selecting an appropriate guidance scale.

C. Scalability Analysis

We evaluate the scalability of CFPG with respect to the number of robots N. Execution of CFPG involves solving an ODE over T steps. At each time step, the computational cost includes one forward pass of the flow network v_θ, and gradient computations for M execution objectives. Since the policy is decentralized, the network inference time is $O(1)$ with respect to the number of robots N due to parallelization. For execution objectives, the gradient computation complexity depends on the objective type: local objectives (e.g., speed limit) are $O(1)$, while team objectives (e.g., collision avoidance) can scale with the number of neighbors K, i.e., $O(K)$, where $K \leq N$ depending on the communication radius. The hierarchical gradient projection involves vector operations on M gradients, scaling as $O(M^2)$ due to worst-case pairwise gradient projections. Thus, the total time complexity per robot per control step is $O(T \cdot (M \cdot K + M^2))$. Given that T (typically ≤ 10) and M (≤ 2 in our experiments) are small constants, CFPG remains computationally efficient and scalable for real-time control.

D. Communication Bandwidth Analysis

From a communication perspective, certain execution objectives require global knowledge:

- Local Objectives (e.g., Speed Limit, Spacing): Only require state information of the ego robot, and can be achieved with no communication.
- Team Objectives (e.g., Clustering, Ordering): Typically require broadcasting robot observations to all neighbors, so communication scales as $O(N^2)$ in total bandwidth.

Additionally, for team objectives like formation or clustering, each robot needs to compute gradients with respect to its neighbors' positions. Assuming a state size of S bytes (e.g., 24 bytes for 2D pose + velocity), and a broadcast frequency f, we compute that the bandwidth per agent is $B = f \cdot S \cdot N$. For our system with $N = 5$, this can be supported by the wireless communication network ($\leq$ 10KB/s at $\geq$ 10Hz). We can use local communication (k-nearest) to manage the communication load by reducing communication complexity to $O(KN)$.

VII. Extended Related Work

A. Multi-Agent Reinforcement Learning and Coordination

Multi-Agent Reinforcement Learning (MARL) has become a standard paradigm for solving complex coordination problems where independent agents must learn to act in a shared environment. Standard algorithms typically adopt the Centralized Training with Decentralized Execution (CTDE) framework. Notably, MAPPO [4] demonstrated that Proximal Policy Optimization (PPO), when adapted with a centralized value function, can achieve state-of-the-art performance on cooperative benchmarks, often surpassing specialized value-factorization methods like QMIX [11]. However, these approaches primarily focus on optimizing a single scalar reward, often failing to capture the nuances of tasks requiring complex trade-offs between competing team and individual goals.

To address the limitations of scalar rewards and improve sampling efficiency in generative MARL, recent works have explored flow-based formulations. For instance, MAC-Flow [12] introduces a flow-matching framework that learns a joint behavioral policy and distills it into efficient decentralized one-step policies, significantly reducing inference latency compared to diffusion-based baselines. Similarly, OM^{2}P [13] proposes an offline multi-agent mean-flow policy that enables one-step action generation by integrating mean-flow matching with Q-function supervision, avoiding the high computational cost of iterative sampling.

Research into social dilemmas and mixed-motive games has further highlighted the difficulty of aligning agent behaviors when individual incentives conflict with collective welfare. Several works [14] and [15] explore how inequity aversion and sequential social dilemmas impact cooperation. Recent approaches have proposed mechanisms to balance altruism and self-interest based on empathy [16] or to align individual and collective objectives through intrinsic rewards [17]. While these methods improve coordination during training, they typically bake the learned preferences into a static policy. Functional Reward Encodings (FRE) [18] attempt to mitigate this rigidity by learning functional representations of arbitrary tasks for zero-shot transfer, although they still rely on specific reward-annotated samples to adapt. In contrast, our CFPG approach decouples the learning of the collaborative manifold from the specific preference vector, allowing for the dynamic re-weighting of objectives (such as shifting from aggressive to safe behaviors) without retraining.

B. Multi-Objective Optimization and Gradient Manipulation

Real-world deployment often requires satisfying multiple, often conflicting, objectives. Multi-Objective MARL (MO-MARL) addresses this by seeking a set of Pareto-optimal policies. Traditional approaches use scalarization functions (e.g., linear or Chebyshev) to convert vector rewards into scalar signals [19], [20], or attempt to approximate the entire Pareto front [21], [22]. However, scalarization requires pre-defined weights during training, limiting flexibility.

A critical challenge in multi-objective learning is *gradient conflict*, where improving one objective degrades another. Gradient manipulation techniques have been developed to mitigate this interference. PCGrad [7] projects conflicting gradients onto each other's normal planes to prevent destructive interference in multi-task learning. Similarly, CAGrad [23] optimizes for the worst-case objective improvement, while GradNorm [24] normalizes gradient magnitudes to balance learning rates. MGDA [25] provides a theoretically rigorous method for finding a common descent direction for all objectives. These techniques have recently been adapted to MARL to ensure fairness [8] and enhance safety [26]. Our work extends these concepts to the inference phase of generative models. Unlike previous works that apply gradient projection to train fixed network weights, CFPG applies hierarchical gradient projection to the *generative flow* during execution. This allows us to resolve conflicts between the learned collaboration manifold (nominal objective) and novel, unseen constraints (execution objectives) in real-time.

C. Generative Models for Robot Control

Generative modeling has revolutionized policy representation in robotics by enabling the capture of multi-modal distributions, a key limitation of unimodal Gaussian policies used in standard PPO. Diffusion models [27]–[30] have been successfully

adapted for imitation learning in robotics, commonly termed as Diffusion Policy [10], [31]–[33]. These policies learn to denoise random noise into valid action trajectories, demonstrating superior performance in capturing human-like behavior and handling multi-modality. BeyondMimic [34] leverages guided diffusion to enable humanoid robots to learn agile motions and adapt to unseen tasks via test-time optimization. To address the inference bottleneck of diffusion, VITA [35] introduces a noise-free flow matching policy that directly maps visual latents to actions, eliminating the need for iterative conditioning. Moreover, General Policy Composition (GPC) [36] demonstrates that pre-trained diffusion and flow policies can be composed at test-time to yield superior performance, offering a training-free paradigm for combining diverse skills. In the context of offline RL, Policy-Guided Diffusion (PGD) [37] generates synthetic experience guided by a target policy to bridge the gap between behavior and target distributions.

Flow matching [6], [38] has emerged as a recent alternative to diffusion that offers faster, simulation-free training and deterministic inference paths without complex noise scheduling. Recent works have begun applying flow matching to control problems. Flow Policy Optimization (FPO) [1] introduced flow matching for single-agent RL, while others have explored action flow matching for continual learning [39], [40] and general robot control [41]. PolicyFlow [42] extends this to on-policy RL by approximating importance ratios via velocity field variations, enabling stable PPO-style updates with continuous normalizing flows. Furthermore, FM-IRL [43] addresses the lack of online interaction in flow-based policies by integrating reward modeling and policy regularization to improve generalization. However, most existing flow-based control methods for multi-robot systems rely on offline datasets or behavior cloning. CFPG introduces *Multi-Agent Flow Policy Optimization (MAFPO)*, a fully on-policy algorithm that trains multi-agent flow policies without expert demonstrations, addressing the data scarcity problem in multi-robot domains.

D. Guidance, Safety, and Adaptation

A unique advantage of generative policies is the ability to modify the generation process using the guidance property. Classifier-free guidance [44] and classifier guidance [45] are standard techniques used in computer vision. In control and optimization, recent works have formalized guidance as a gradient-based optimization process [5], [46], [47]. CFGRL [48] reinterprets classifier-free guidance as a policy improvement operator which allows for controllable optimization of offline policies without explicit value function learning. Independently, another work [49] proposes learning state-dependent guidance weights to better approximate target conditional distributions. Beyond diffusion, Policy Gradient Guidance (PGG) [50] demonstrates that guidance mechanisms can be adapted to standard on-policy RL methods to modulate behavior at test-time without retraining.

Training-free guidance methods [51], [52] allow for the addition of constraints without retraining the base model. Specifically, ParetoFlow [53] explores guiding flows toward the Pareto front of multiple objectives. OC-Flow [54] introduces a training-free framework for guided flow matching using optimal control, applicable even to complex geometries like $SO(3)$. DriftLite [55] introduces a lightweight, training-free approach to steer inference dynamics by actively controlling particle drift, reducing variance in particle-based guidance. In the realm of safety, Safety-Guided Flow [56] unifies negative guidance methods to suppress unsafe generations, employing a Maximum Mean Discrepancy potential to repel trajectories from unsafe regions. Additionally, Test-Time Reinforcement Learning (TTRL) [57] leverages test-time compute to modify outputs using reward estimation on unlabeled data, highlighting the growing trend of inference-time adaptation. However, none of these works currently deal with multi-robot systems in the context of online RL.

In the domain of safe reinforcement learning, constraints are typically handled via rigid mechanisms such as shielding [58], [59] or constrained policy optimization [60], [61]. While effective for ensuring hard safety, shielding can often result in overly conservative behaviors that can hinder success in tasks. In contrast, CFPG offers a more flexible alternative by incorporating execution objectives (which can include safety) using guidance within the flow matching ODE. By combining this with our hierarchical gradient projection, we show that safety through guidance can still be consistent with the underlying collaborative manifold learned by the agents.

VIII. Limitations and Future Work

The CFPG approach opens up several exciting directions for future research that extend the capabilities of multi-robot teams. One direction is adapting the CFPG approach to heterogeneous teams with possibly different kinematics, dynamics, and sensor modalities. Although our approach focused on homogeneous agents, the underlying flow matching formulation is generalizable and can be extended to heterogeneous teams (e.g., unmanned aerial and ground vehicles), which can take advantage of their unique strengths for more complex tasks. Additionally, we can incorporate varying communication topologies for practical deployment and real-world applicability. Currently, CFPG assumes we can optimize global team objectives that require full communication or position broadcasting; future work could explore decentralized gradient estimation techniques or graph-based consensus protocols to achieve robust collaboration in communication-constrained environments.

Another possible direction is the integration of hard safety constraints directly into the generative process. While CFPG currently guarantees that execution-time guidance remains consistent with the learned collaborative manifold via hierarchical gradient projection, future iterations could explicitly incorporate theoretically guaranteed constraints such as control barrier

functions (CBFs) into the flow vector field guidance directly. This can offer an additional layer of strict guarantees for safety-critical systems while still taking advantage of modern generative models. Furthermore, we have potential to improve the robustness of the guidance mechanism, especially regarding execution objectives that may fundamentally conflict with the pretrained collaborative manifold. Developing mechanisms to detect such constraints (especially for safety) could trigger higher-level replanning or explicit relaxation of the velocity field as needed.

Lastly, while CFPG operates in the paradigm of centralized training with decentralized execution, exploring fully decentralized training paradigms is another promising direction. Finding a way to replace (or approximate) the centralized value function during training can enable the deployment of more robust learning-based flow policies in scenarios where global state information is unavailable or difficult to aggregate. Addressing these new challenges will further solidify the role of generative flow policies for adaptive, scalable, and robust multi-robot collaboration.

REFERENCES

[1] D. McAllister, S. Ge, B. Yi, C. M. Kim, E. Weber, H. Choi, H. Feng, and A. Kanazawa, "Flow Matching Policy Gradients," in *International Conference on Learning Representations*, 2026.

[2] Z. Xie, Q. Zhang, F. Yang, M. Hutter, and R. Xu, "Simple Policy Optimization," in *International Conference on Machine Learning*, 2025.

[3] D. P. Kingma and R. Gao, "Understanding Diffusion Objectives as the ELBO with Simple Data Augmentation," in *Neural Information Processing Systems*, 2023.

[4] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games," in *Neural Information Processing Systems*, 2022.

[5] R. Feng, C. Yu, W. Deng, P. Hu, and T. Wu, "On the Guidance of Flow Matching," in *International Conference on Machine Learning*, 2025.

[6] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat, "Flow Matching Guide and Code," *arXiv preprint arXiv:2412.06264*, 2024.

[7] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn, "Gradient Surgery for Multi-Task Learning," in *Neural Information Processing Systems*, 2020.

[8] W. Kim and K. Sycara, "Fair Cooperation in Mixed-Motive Games via Conflict-Aware Gradient Adjustment," in *Neural Information Processing Systems*, 2025.

[9] M. Bettini, R. Kortvelesy, J. Blumenkamp, and A. Prorok, "VMAS: A Vectorized Multi-Agent Simulator for Collective Robot Learning," in *International Symposium on Distributed Autonomous Robotic Systems*, 2022.

[10] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Visuomotor Policy Learning via Action Diffusion," in *Robotics: Science and Systems*, 2023.

[11] T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," in *International Conference on Machine Learning*, 2018.

[12] D. Lee, D. Lee, and A. Zhang, "Multi-agent coordination via flow matching," in *International Conference on Learning Representations*, 2026.

[13] Z. Li, X. Wang, H. Zhong, and L. Huang, "Om^{2}p: Offline multi-agent mean-flow policy," in *International Conference on Autonomous Agents and Multiagent Systems*, 2026.

[14] J. Z. Leibo, V. Zambaldi, and M. Lanctot, "Multi-agent Reinforcement Learning in Sequential Social Dilemmas," in *International Conference on Autonomous Agents and Multiagent Systems*, 2017.

[15] E. Hughes, J. Z. Leibo, M. Phillips, K. Tuyls, E. Dueñez-Guzman, A. García Castañeda, I. Dunning, T. Zhu, K. McKee, R. Koster, H. Roff, and T. Graepel, "Inequity aversion improves cooperation in intertemporal social dilemmas," in *Neural Information Processing Systems*, 2018.

[16] F. Kong, Y. Huang, S.-C. Zhu, S. Qi, and X. Feng, "Learning to Balance Altruism and Self-interest Based on Empathy in Mixed-Motive Games," in *Neural Information Processing Systems*, 2024.

[17] Y. Li, W. Zhang, J. Wang, and S. Zhang, "Aligning Individual and Collective Objectives in Multi-Agent Cooperation," in *Neural Information Processing Systems*, 2024.

[18] K. Frans, S. Park, P. Abbeel, and S. Levine, "Unsupervised zero-shot reinforcement learning via functional reward encodings," in *International Conference on Machine Learning*, 2024.

[19] D. M. Roijers, P. Vamplew, S. Whiteson, and R. Dazeley, "A Survey of Multi-Objective Sequential Decision-Making," *Journal of Artificial Intelligence Research*, vol. 48, pp. 67–113, 2013.

[20] A. Abels, D. Roijers, T. Lenaerts, A. Nowé, and D. Steckelmacher, "Dynamic Weights in Multi-Objective Deep Reinforcement Learning," in *International Conference on Machine Learning*, 2019.

[21] K. V. Moffaert and A. Nowé, "Multi-Objective Reinforcement Learning using Sets of Pareto Dominating Policies," *Journal of Machine Learning Research*, vol. 15, no. 107, pp. 3663–3692, 2014.

[22] R. Yang, X. Sun, and K. Narasimhan, "A Generalized Algorithm for Multi-Objective Reinforcement Learning and Policy Adaptation," in *Neural Information Processing Systems*, 2019.

[23] B. Liu, X. Liu, X. Jin, P. Stone, and Q. Liu, "Conflict-Averse Gradient Descent for Multi-task learning," in *Neural Information Processing Systems*, 2021.

[24] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich, "GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks," in *International Conference on Machine Learning*, 2018.

[25] J.-A. Désidéri, "Multiple-gradient descent algorithm (MGDA) for multiobjective optimization," *Comptes Rendus. Mathématique*, vol. 350, no. 5-6, pp. 313–318, 2012.

[26] S. Zhang, O. So, M. Black, Z. Serlin, and C. Fan, "Solving Multi-Agent Safe Optimal Control with Distributed Epigraph Form MARL," in *Robotics: Science and Systems*, 2025.

[27] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Neural Information Processing Systems*, 2020.

[28] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-Based Generative Modeling Through Stochastic Differential Equations," in *International Conference on Learning Representations*, 2021.

[29] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *International Conference on Learning Representations*, 2021.

[30] L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui, and M.-H. Yang, "Diffusion Models: A Comprehensive Survey of Methods and Applications," *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–39, 2023.

[31] E. Ng, Z. Liu, and M. Kennedy, "Diffusion Co-Policy for Synergistic Human-Robot Collaborative Tasks," *IEEE Robotics and Automation Letters*, vol. 9, no. 1, pp. 215–222, 2024.

[32] A. Sridhar, D. Shah, C. Glossop, and S. Levine, "NoMaD: Goal Masked Diffusion Policies for Navigation and Exploration," in *IEEE International Conference on Robotics and Automation*, 2024.

[33] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations," in *Robotics: Science and Systems*, 2024.

[34] Q. Liao, T. E. Truong, X. Huang, Y. Gao, G. Tevet, K. Sreenath, and C. K. Liu, "Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion," *arXiv preprint arXiv:2508.08241*, 2025.

[35] D. Gao, B. Zhao, A. Lee, I. Chuang, H. Zhou, H. Wang, Z. Zhao, J. Zhang, and I. Soltani, "Vita: Vision-to-action flow matching policy," in *International Conference on Learning Representations*, 2026.

[36] J. Cao, Y. Huang, H. Guo, R. Zhang, M. Nan, W. Mai, J. Wang, H. Cheng, J. Sun, G. Han, W. Zhao, Y. Guo, Q. Zheng, X. Li, C. Song, P. Luo, and A. Luo, "Compose your policies! improving diffusion-based or flow-based robot policies via test-time distribution-level composition," in *International Conference on Learning Representations*, 2025.

[37] M. T. Jackson, M. T. Matthews, C. Lu, B. Ellis, J. N. Foerster, and S. Whiteson, "Policy-guided diffusion," in *Reinforcement Learning Conference*, 2024.

[38] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow Matching for Generative Modeling," in *International Conference on Learning Representations*, 2023.

[39] A. Murillo-Gonzalez and L. Liu, "Action Flow Matching for Continual Robot Learning," in *Robotics: Science and Systems*, 2025.

[40] M. M. Sun, A. Pinosky, and T. Murphey, "Flow Matching Ergodic Coverage," in *Robotics: Science and Systems*, 2025.

[41] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, "π_0: A Vision-Language-Action Flow Model for General Robot Control," in *Robotics: Science and Systems*, 2025.

[42] S. Yang, B. Liu, and H. Chen, "Policyflow: Policy optimization with continuous normalizing flow in reinforcement learning," in *International Conference on Learning Representations*, 2026.

[43] Z. Wan, J. Wu, X. Yu, C. Zhang, M. Lei, B. An, and I. Tsang, "Fm-irl: Flow-matching for reward modeling and policy regularization in reinforcement learning," *arXiv preprint arXiv:2510.09222*, 2025.

[44] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," in *Neural Information Processing Systems Workshop on Deep Generative Models and Downstream Applications*, 2021.

[45] P. Dhariwal and A. Nichol, "Diffusion Models Beat GANs on Image Synthesis," in *Neural Information Processing Systems*, 2021.

[46] Y. Guo, H. Yuan, Y. Yang, M. Chen, and M. Wang, "Gradient Guidance for Diffusion Models: An Optimization Perspective," in *Neural Information Processing Systems*, 2024.

[47] P. Marion, A. Korba, P. Bartlett, M. Blondel, V. D. Bortoli, A. Doucet, F. Llinares-Lopez, C. Paquette, and Q. Berthet, "Implicit Diffusion: Efficient optimization through stochastic sampling," in *International Conference on Artificial Intelligence and Statistics*, 2025.

[48] K. Frans, S. Park, P. Abbeel, and S. Levine, "Diffusion guidance is a controllable policy improvement operator," *arXiv preprint arXiv:2505.23458*, 2025.

[49] A. Galashov, A. Pokle, A. Doucet, A. Gretton, M. Delbracio, and V. De Bortoli, "Learn to guide your diffusion model," in *International Conference on Learning Representations*, 2026.

[50] J. Qi, H. Tang, and Z. Zhu, "Policy gradient guidance enables test time control," *arXiv preprint arXiv:2510.02148*, 2025.

[51] H. Ye, H. Lin, J. Han, M. Xu, S. Liu, Y. Liang, J. Ma, J. Zou, and S. Ermon, "TFG: Unified Training-Free Guidance for Diffusion Models," in *Neural Information Processing Systems*, 2024.

[52] A. Bansal, H.-M. Chu, A. Schwarzschild, S. Sengupta, M. Goldblum, J. Geiping, and T. Goldstein, "Universal Guidance for Diffusion Models," in *International Conference on Learning Representations*, 2024.

[53] Y. Yuan, C. Pal, and X. Liu, "ParetoFlow: Guided Flows in Multi-Objective Optimization," in *International Conference on Learning Representations*, 2025.

[54] L. Wang, C. Cheng, Y. Liao, Y. Qu, and G. Liu, "Training free guided flow matching with optimal control," in *International Conference on Learning Representations*, 2025.

[55] Y. Ren, W. Gao, L. Ying, G. M. Rotskoff, and J. Han, "Driftlite: Lightweight drift control for inference-time scaling of diffusion models," in *International Conference on Learning Representations*, 2026.

[56] M. Kim, Y.-H. Kim, and M. Park, "Safety-guided flow: A unified framework for negative guidance in safe generation," in *International Conference on Learning Representations*, 2026.

[57] Y. Zuo, K. Zhang, L. Sheng, X. Zhu, H. Li, Y. Zhang, E. Hua, B. Qi, Y. Sun, N. Ding, and B. Zhou, "Ttrl: Test-time reinforcement learning," in *Neural Information Processing Systems*, 2025.

[58] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe Reinforcement Learning via Shielding," in *AAAI Conference on Artificial Intelligence*, 2018.

[59] I. ElSayed-Aly, S. Bharadwaj, C. Amato, R. Ehlers, U. Topcu, and L. Feng, "Safe Multi-Agent Reinforcement Learning via Shielding," in *International Conference on Autonomous Agents and Multiagent Systems*, 2021.

[60] T.-Y. Yang, J. Rosca, K. Narasimhan, and P. J. Ramadge, "Projection-Based Constrained Policy Optimization," in *International Conference on Learning Representations*, 2020.

[61] S. Gu, L. Shi, Y. Ding, A. Knoll, C. Spanos, A. Wierman, and M. Jin, "Enhancing Efficiency of Safe Reinforcement Learning via Sample Manipulation," in *Neural Information Processing Systems*, 2024.